
The Telolexic Method: A Universal Inverse-Recursive Audit Protocol (TAP) for Multimodal Generative Systems and Software State Verification

Udimudi Naga Raju

www.pixel9.in

India

raju9@pixel9.in

December 20, 2025

Abstract

The dominant paradigm in contemporary computing—ranging from Large Language Models (LLMs) and Diffusion Video Generators to Event-Driven Mobile Architectures—relies fundamentally on **Forward-Chaining Probability**. In this model, system state t_n is derived linearly from t_0 . While computationally efficient for generation, this unidirectional flow creates a "Verification Blind Spot," leading to critical reliability failures: logical hallucinations in text, physical inconsistencies in video (such as object permanence failure), and unrecoverable state-loss in mobile applications.

This paper introduces **The Telolexic Method**, a novel verification framework implemented via the **Telolexic Audit Protocol (TAP)**. Deriving its nomenclature from the Greek *telos* (end) and the Sanskrit concept of *Vilomapatha* (reverse reading), TAP functions as a model-agnostic governance layer. It treats any generated output or crash state as a fixed "Terminal Anchor" and performs an **Inverse-Recursive Audit** ($N \rightarrow 1$) to verify causal necessity.

We detail the application of TAP across four distinct domains: (1) **The Telolexic Pre-verified Transformer (TPT)** for zero-hallucination AI; (2) **Visual TAP**, a computer vision protocol solving the five major classes of generative video errors (Flicker, Physics, Anatomy, Inventory, and Drift); (3) **The Viloma Crash Tracker** for Android Lifecycle management; and (4) **Viloma-Cryptography**, a terminal-dependent security primitive. Experimental results demonstrate that TAP identifies causal breaks in video streams with 85% accuracy, significantly outperforming forward-tracking metrics.

Keywords— *Telolexia, Vilomapatha, Telolexic Audit Protocol (TAP), Visual TAP, Inverse-Causal Tracking, Android Lifecycle, Agentic Supervision, AI Safety.*

I. INTRODUCTION

A. The Linearity Bias in Computing

Since the inception of the Turing Machine, software execution has been conceptualized as a forward-moving tape. In modern Artificial Intelligence, this "Linearity Bias" is codified in the Transformer architecture [1], which predicts the next token $t_{(i+1)}$ based on the probability distribution of preceding tokens $t_0 \dots t_i$. Similarly, video generation models predict pixel clusters frame-by-frame.

While this approach excels at **Creativity** (generating plausible continuations), it is structurally incapable of **Veracity** (ensuring logical consistency). A model generating a mystery novel forward may write a conclusion that contradicts a clue dropped in Chapter 1, simply because the "context window" degraded or the probabilistic weight shifted.

B. The Telolexic Hypothesis

We propose that Truth is a function of the End, not the Beginning.

In deductive logic, a conclusion is valid only if the premises necessitate it. However, in auditing a generated narrative or a software crash, the "End State" (S_{term}) is the only objective reality we possess. Therefore, the most rigorous method of verification is not to re-generate forward, but to audit backward.

This paper formally defines **Telolexia** as the computational science of **Inverse-Recursive State Verification**.

II. THEORETICAL FRAMEWORK

A. Mathematical Formulation of Causal Chains

Let a sequential system **D** be defined as a discrete vector of states:

$$\mathbf{D} = \{S_0, S_1, S_2, \dots, S_n\}$$

In a standard generative model (Forward Chaining), the state $S_{(i+1)}$ is produced by a function **G**:

$$S_{(i+1)} = \mathbf{G}(S_i, \theta)$$

Where θ represents model parameters.

The **Telolexic Method** introduces a verification function **V** that operates inversely. For a state $S_{(i+1)}$ to be considered "Telolexically Sound," its existence must mandate the presence of a prerequisite set **{P}** in S_i :

$$V(S_{(i+1)} \rightarrow S_i) = 1 \text{ (if P exists in } S_i)$$

$$V(S_{(i+1)} \rightarrow S_i) = 0 \text{ (if P is missing from } S_i \text{ — Hallucination)}$$

B. The Vilomapatha Protocol

The implementation of this theory is the **Vilomapatha (Reverse Path) Protocol**.

- Terminal Anchoring:** Lock S_n as immutable.
- Reverse Traversal:** Iterate pointer **k** from **n** down to **0**.
- Causal Dependency Check:** At each step **k**, identify dependencies required by S_k and query $S_{(k-1)}$ for their existence.

III. APPLICATION I: TEXT & AI AGENTS (THE TPT)

A. The Telolexic Pre-verified Transformer (TPT)

Standard GPT models output tokens immediately. The TPT architecture introduces a **Latent Buffer**.

1. **Generation Phase:** The model generates the full response into a hidden buffer.
2. **Audit Phase:** A secondary "Telolexic Head" (or separate Agent) reads the buffer from the last sentence to the first.
3. **Validation:**
 - *Claim:* "Therefore, X is true."
 - *Audit:* Scan previous paragraphs. Is there a premise that supports X?
 - *Action:* If logical support is missing, the sentence is pruned before display.

B. Agentic Supervision (The Antigravity Workflow)

In autonomous agent environments (e.g., Google Antigravity), we utilize a dual-agent workflow:

- **Agent A (The Creative):** High-temperature model optimized for fluency.
- **Agent B (The Telolexist):** Low-temperature model optimized for logic.
- **Protocol:** Agent B receives the output of Agent A. It parses the text into claims and verifies them in reverse order. Only verified claims are released to the user.

IV. APPLICATION II: VISUAL TAP (GENERATIVE VIDEO)

The most commercially urgent application of Telolexia is in AI Video Generation. Current models suffer from "dream-like" physics where objects morph or vanish. We classify these errors into five categories and provide Telolexic solutions for each.

A. Problem 1: Temporal Flicker (Texture Drift)

- *The Phenomenon:* An object (e.g., a shirt) changes color or texture across frames due to context loss.
- *The Telolexic Solution (Retro-Causal Texture Locking):*
 - The algorithm identifies the object in the **Final Frame (t_end)**.
 - It treats this final texture as the "Ground Truth."
 - It performs **Reverse Optical Flow** to map this texture back to **t_start**.
 - Any deviation in intermediate frames is overwritten by the warped texture from **t_end**, ensuring absolute temporal consistency.

B. Problem 2: Visual Physics Hallucinations

- *The Phenomenon:* Shadows pointing towards the sun; reflections showing the wrong object.
- *The Telolexic Solution (Inverse-Optical Grounding):*
 - **Step 1:** Object detection identifies "Effect Artifacts" (Shadows, Reflections) in **S_term**.
 - **Step 2:** Ray-tracing vectors are calculated *from* the shadow *to* the potential source.
 - **Step 3:** If the vector intersects with a light source in the 3D scene estimation, the frame is valid. If it points to empty space, the frame is rejected.

C. Problem 3: Anatomical Distortion (The "Spaghetti Hand")

- *The Phenomenon:* Generation of hands with >5 fingers or extra limbs.
- *The Telolexic Solution (Skeletal Regression):*
 - Instead of counting fingers forward, we trace **Bone Chains** backward.
 - Starting from a fingertip pixel in **S_term**, we trace the skeletal path to the wrist node.
 - **Constraint:** If the number of distinct chains converging on a single wrist node > 5, the entity is flagged as biologically impossible.

D. Problem 4: Object Permanence Failure

- *The Phenomenon:* An object is occluded (hidden) and fails to reappear, or appears out of nowhere ("Pop-in").
- *The Telolexic Solution (The Viloma Inventory):*
 - **Inventory Creation:** At t_0 , the system logs all active objects $\{O_1, O_2 \dots\}$.
 - **Reverse Audit:** At t_n , the system checks for the presence of $\{O\}$.
 - **Verification:** If O_1 is missing at t_n , the auditor scans the reverse stream for a valid "Exit Event" (e.g., object dropped, consumed, or moved off-screen). If no Exit Event exists, the disappearance is flagged as a "Mass Conservation Error."

E. Problem 5: Prompt Drift

- *The Phenomenon:* The video ending diverges entirely from the user's initial prompt.
- *The Telolexic Solution (Terminal State Anchoring):*
 - The verification process begins by comparing the **Last Frame** against the **Text Prompt**.
 - If the Last Frame does not contain the required elements (e.g., "A bike"), the audit fails immediately (Fail-Fast), saving computational resources on checking intermediate frames.
 -

V. APPLICATION III: MOBILE SOFTWARE ENGINEERING

A. The Android Lifecycle Problem

Android applications operate as state machines. Crashes often occur when the system kills a background process (to save memory) and attempts to restore it in a "Zombie State"—where the UI exists, but the underlying data variables are null.

B. The Viloma Crash Tracker Algorithm

We introduce a custom **UncaughtExceptionHandler** that utilizes a fixed-size **Stack<State>**.

1. **Logging:** Every Lifecycle event (onCreate, onPause, User Input) is pushed to the stack.
2. **Crash Interception:** On crash, execution is suspended.
3. **Inverse Audit:** The handler reads the stack from Top (Crash) to Bottom.
4. **Gap Detection:** It looks for the pattern: **Process Death** → **Restoration** → **Variable Access**.
5. **Diagnosis:** It pinpoints the exact variable that was not re-initialized during the Restoration phase, distinguishing logic bugs from system-induced state loss.

VI. APPLICATION IV: VILOMA CRYPTOGRAPHY

A. Terminal-Dependent Encryption

Standard stream ciphers allow decryption to begin as soon as the first byte is received. While fast, this allows malicious actors to inject packets if they compromise the stream mid-transmission.

B. The Viloma-Hash

We propose a hashing algorithm where the **Initialization Vector (IV)** for the header is derived from the **Checksum of the Footer**.

Key_header = $\Phi(\text{Block_N})$

This enforces an "All-or-Nothing" property: the receiver cannot read the beginning of the message until the entire stream has been received and the end verified. This is critical for high-security atomic transactions.

VII. EXPERIMENTAL METHODOLOGY & RESULTS

To validate the Telolexic Method, we constructed a "**Telolexic Lab**" environment using Open Source tools (Llama-3 via Ollama, OpenCV, and Android Studio).

A. Experiment 1: Video Hallucination Detection

- **Dataset:** 50 AI-generated videos containing known glitches (objects appearing/disappearing) and 50 Real-world control videos.

- **Method:** We implemented the **Visual TAP** script using OpenCV's CSRT Tracker in reverse mode ($t_n \rightarrow t_0$).
- **Results:**
 - Standard Forward Optical Flow acceptance rate for glitched videos: **88%** (Failed to detect glitch).
 - Visual TAP Rejection rate for glitched videos: **85%** (Successfully detected glitch).
 - *Conclusion:* The inverse tracking requirement effectively filters out "pop-in" hallucinations.

B. Experiment 2: Text Logic Verification

- **Setup:** We prompted Llama-3 with complex historical counter-factuals designed to induce hallucinations.
- **Method:** An Agentic Supervisor (Agent B) audited the responses using the Viloma Protocol.
- **Results:** The Telolexic Supervisor identified and flagged logical inconsistencies in 15% of responses that standard "perplexity" metrics had rated as high-quality.

VIII. DISCUSSION & FUTURE SCOPE

The transition from **Generative AI** to **Verifiable AI** is the defining challenge of the next decade. The Telolexic Method provides a standardized protocol for this transition.

A. Implications for AGI

As AI systems become more autonomous, the risk of "Compound Error" (where one small hallucination cascades into a massive failure) increases. A recursive, backward-looking audit layer acts as a "Subconscious Superego," constantly checking generated thoughts against established reality.

B. Future Work

We are currently developing a lightweight "**Telolexic Kernel**" for mobile devices—a background service that applies these principles to OS-level resource management, potentially eliminating "App Not Responding" (ANR) errors entirely.

IX. CONCLUSION

We have presented the **Telolexic Method**, a comprehensive framework for Inverse-Recursive State Verification. By decoupling the *creation* of data (Forward) from the *verification* of data (Backward), we address the fundamental "Linearity Bias" that plagues modern computing. Whether ensuring that a generated video obeys the laws of physics, or that an Android app survives process death, the solution lies in the same direction: **Vilomapatha**—the path backwards.

X. REFERENCES

- [1] Vaswani, A., et al. (2017). "Attention Is All You Need." Advances in Neural Information Processing Systems.
- [2] U Naga Raju. (2025). "The Telolexic Manifesto: Inverse Narrative Deduction."
- [3] Open Source Computer Vision Library (OpenCV). "CSRT Tracking Algorithm & Optical Flow."
- [4] Android Developers Documentation. "Activity Lifecycle & State Restoration."
- [5] Ancient Vedic Linguistics. "The Structure of Viloma Paatha Recitation."
- [6] Google DeepMind. "Agentic Workflows and Antigravity IDE Documentation."
- [7] NIST Special Publication 800-22. "Randomness Testing and Stream Integrity."